Pareto Analysis SQL Project: Petshop dataset

By Alexandra Curran

Table of Contents

Project Summary 2
Pareto Analysis Explanation 2
Dataset Exploration 2
Methodology 3
Target Threshold Analysis 8
Conclusion 10
Possible Next Steps 11
Query Data Dictionary: 11

Figure 1: Original Petshop Dataset 3
Figure 2: Query to find total sales 4
Figure 3: Query to find total number of customers. 4
Figure 4: Query to create view 1: 'sales_v1' 4
Figure 5: 'sales_v1' findings. 5
Figure 6: Query to create 'sales_v2'. 5
Figure 7: 'sales_v2' findings. 6
Figure 8: Query uses window functions to list each customer once and assign them a sequential rank by CustomerRevenue (highest to lowest), while also computing cumulative revenue, total counts, and total revenue across all customers. 6
Figure 9: 'sales_v3' findings. 7
Figure 10: Cuml_Sales_Share and Cuml_PCT_Customers reached 100% (1.0). 7
Figure 11: We can also see now that about 80% of sales come from roughly 57% of customers. 8
Figure 12: Query and results of cumulative percent of customers from 80% of the sales. 150/261 = 50% 8
Figure 13: Query and results of cumulative percent of customers from 50% of the sales. 79/261 = 30% 9
Figure 14: Figure 13: Query and results of cumulative percent of customers from 20% of the sales. 28/261 = 11% 10

Project Summary

This is a Pareto Analysis project using a dataset from a pet shop. I utilized SQL in Google BigQuery to create the project repository. The objective is to determine what proportion of customers' accounts for 20%, 50%, and 80% of total revenue, and to use those findings to evaluate how concentrated revenue is across the customer base.

Pareto Analysis Explanation

Pareto analysis is a prioritization method used to identify the relatively small set of contributors that account for a disproportionately large share of an outcome. It is commonly associated with the 80/20 guideline, which suggests that 80% of effects often come from 20% of causes, although the exact distribution varies by dataset and business context.

In a business setting, Pareto analysis helps focus attention on the factors that matter most for performance, risk, or value creation. In this project, the analysis is applied to customer revenue to identify how much of the pet shop's revenue is generated by the highest-spending customers.

It is important to note that Pareto analysis identifies where to look first, but it does not determine the corrective or strategic action on its own. Interpretation should therefore be supported by business context, domain knowledge, and professional judgment.

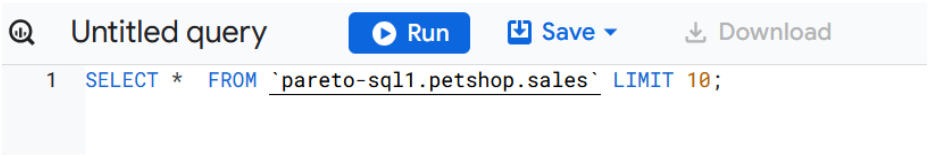
Dataset Exploration

The dataset was first reviewed to identify the available fields and confirm the structure of the source data. This step established the fields needed to calculate customer-level sales and cumulative revenue contribution.

Initial review showed that one invoice number can contain more than one stock code, while each invoice is associated with only one customer ID. This distinction is important because revenue must ultimately be aggregated at the customer level for Pareto analysis.

Dataset Dictionary		
Field name	**Type**	**Description**
InvoiceNo	String	Tracking number of invoice
StockCode	String	ID code of stock
Description	String	Description of stock
InvoiceDate	Date	Date of invoice
CustomerID	Integer	Customer personal ID number

Quantity	Integer	Number of stock item purchased
UnitPrice	Float	Price of stock item (per each item)
Country	String	Customer's country location



Query results

Chat | Save results | Open in

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	InvoiceNo	StockCode	Description	InvoiceDate	CustomerID	Quantity	UnitPrice	Country
1	INV10000	DOG024	Dog Paw Cleaner Cup	2025-02-27	1000	3	15.79	Germany
2	INV10001	DOG003	Dog Tennis Balls Pack of 3	2025-02-22	1001	5	22.87	United States
3	INV10001	DOG001	Leather Dog Collar Medium	2025-02-22	1001	1	14.74	United States
4	INV10001	DOG017	Fleece Dog Blanket	2025-02-22	1001	5	7.05	United States
5	INV10001	DOG007	Reflective Dog Collar	2025-02-22	1001	5	22.77	United States
6	INV10001	DOG015	Silicone Dog Bowl Collapsible	2025-02-22	1001	5	17.12	United States
7	INV10001	DOG001	Leather Dog Collar Medium	2025-02-22	1001	2	33.92	United States
8	INV10001	DOG011	LED Dog Collar Light	2025-02-22	1001	3	12.21	United States
9	INV10001	DOG025	Personalized Dog Tag	2025-02-22	1001	3	10.08	United States

Figure 1: Original Petshop Dataset

Methodology

The analysis was developed incrementally through a sequence of SQL views in BigQuery. Each view added a new layer of transformation so that the final output could measure cumulative customer and revenue contribution accurately.

View 1:

The first step was to calculate line-level sales by multiplying *Quantity* by *UnitPrice* and to identify the total number of customers in the dataset. These values were combined into an initial view named *sales_v1*.

At this stage, multiple rows still existed for the same customer because a customer could appear on more than one invoice. As a result, the data was not yet suitable for customer-level Pareto analysis.

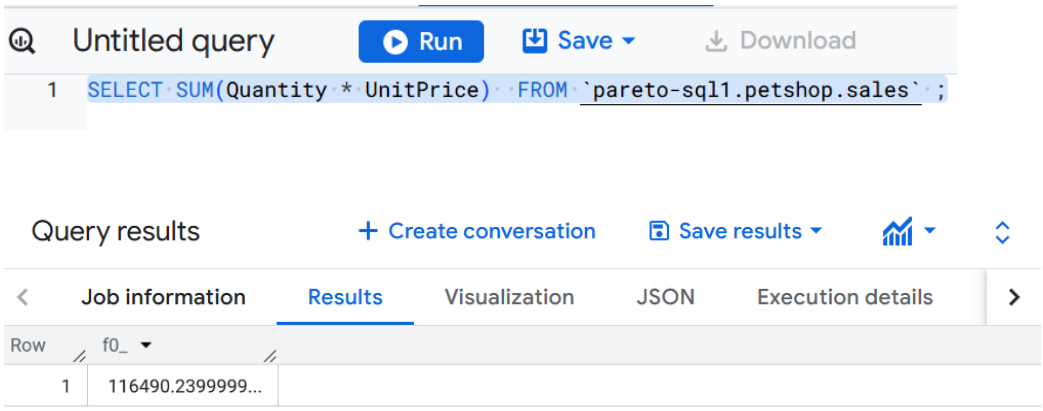


Figure 2: Query to find total sales.

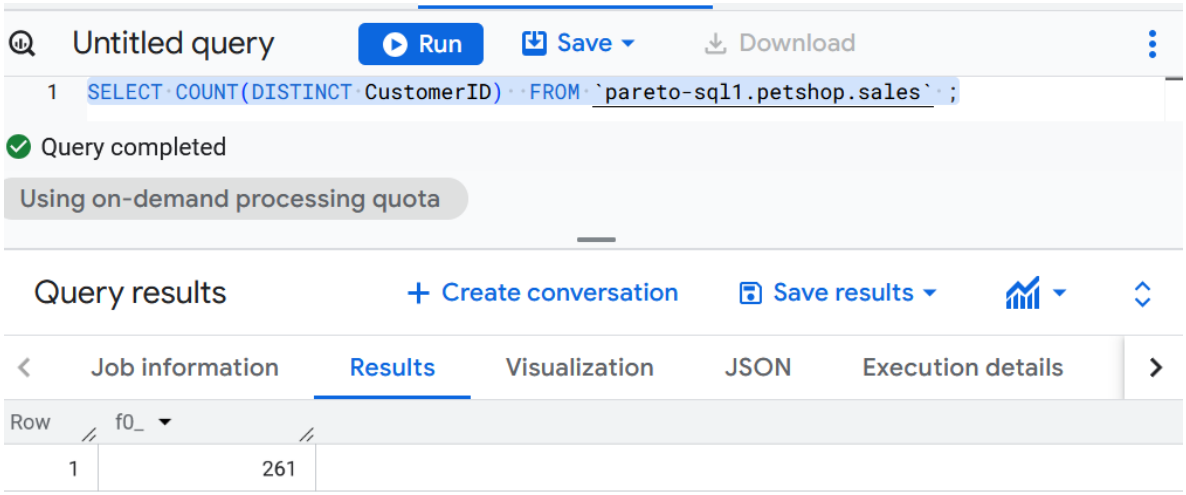


Figure 3: Query to find total number of customers.



Figure 4: Query to create view 1: 'sales_v1'.

```
1  SELECT * FROM `pareto-sql1.petshop.sales_v1` LIMIT 25 ;
```

Query completed

Using on-demand processing quota

Query results + Create conversation Save results

Job information | Results | Visualization | JSON | Execut

Row	CustomerID	Sales
1	1000	47.37
2	1001	114.3500000000...
3	1001	14.74
4	1001	35.25
5	1001	113.85
6	1001	85.60000000000...
7	1001	67.84
8	1001	36.63
9	1001	30.24000000000...
10	1001	20.35
11	1002	109.76
12	1002	28.07
13	1002	207.0499999999...
14	1002	68.4
15	1002	32.44
16	1003	68.44

Figure 5: 'sales_v1' findings.

View 2:

The second view grouped the data by *CustomerID* and summed sales to produce one revenue total per customer. This created a customer-level measure called *CustomerRevenue*, which is the core input for the Pareto analysis.

This transformation in *sales_v2* resolved the duplication issue seen in the first view and ensured that each customer appeared only once in the output.

```
1  CREATE OR REPLACE VIEW `pareto-sql1.petshop.sales_v2`
2  AS
3  SELECT
4    CustomerID,
5    SUM(Sales) AS CustomerRevenue
6  FROM `pareto-sql1.petshop.sales_v1`
7  GROUP BY
8    CustomerID ;
```

Figure 6: Query to create 'sales_v2'.

```
1  SELECT * FROM `pareto-sql1.petshop.sales_v2` LIMIT 25 ;
```

Query completed

Using on-demand processing quota

Query results + Create conversation Sa

Job information | Results | Visualization | JSON

	CustomerID	CustomerRevenue
1	1000	47.37
2	1001	518.85
3	1002	445.7199999999...
4	1003	201.8999999999...
5	1004	239.8
6	1005	756.02
7	1006	626.34
8	1007	523.13
9	1008	219.35
10	1009	358.5399999999...
11	1010	507.0099999999...
12	1011	627.88
13	1012	65.99
14	1013	83.44
15	1014	277.3
16	1015	353.48

Figure 7: 'sales_v2' findings.

View 3:

The third view introduced window functions to rank customers from highest to lowest revenue while retaining row level integrity using a Windows function. It also calculated cumulative revenue, total customer count, and total revenue across the full dataset.

This step in *sales_v3* established the core Pareto logic by enabling the report to track how revenue accumulates as customers are added in descending order of spending.

```
1  CREATE OR REPLACE VIEW `pareto-sql1.petshop.sales_v3`
2  AS
3  SELECT
4    CustomerID,
5    ROW_NUMBER() OVER(ORDER BY CustomerRevenue DESC) AS Cumlative_Customers,
6    COUNT(*) OVER() AS Total_Customers,
7    SUM(CustomerRevenue) OVER(ORDER BY CustomerRevenue DESC
8    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS Cumlative_Revenue,
9    SUM(CustomerRevenue) OVER() AS Total_Revenue
10 FROM `pareto-sql1.petshop.sales_v2` ;
11
```

Figure 8: Query uses window functions to list each customer once and assign them a sequential rank by CustomerRevenue (highest to lowest), while also computing cumulative revenue, total counts, and total revenue across all customers.

```
1  SELECT * FROM `pareto-sql1.petshop.sales_v3` LIMIT 25 ;
```

Query completed

Using on-demand processing quota

Query results + Create conversation Save results

Job information | Results | Visualization | JSON | Execution details

Row	CustomerID	Cumlative_Custo...	Total_Customers	Cumlative_Reven...	Total_Revenue
1	1223	1	261	1100.399999999...	116490.24
2	1163	2	261	2092.95	116490.24
3	1107	3	261	3070.4	116490.24
4	1126	4	261	4026.22	116490.24
5	1118	5	261	4935.83	116490.24
6	1110	6	261	5833.46	116490.24
7	1191	7	261	6730.9	116490.24
8	1133	8	261	7617.849999999...	116490.24
9	1150	9	261	8497.18	116490.24
10	1153	10	261	9363.94	116490.24
11	1051	11	261	10228.93	116490.24
12	1209	12	261	11079.73	116490.24
13	1054	13	261	11917.73	116490.24
14	1046	14	261	12745.75	116490.24
15	1106	15	261	13567.83	116490.24
16	1062	16	261	14385.44	116490.24

Figure 9: 'sales_v3' findings.

View 4:

The fourth view in *sales_v4* extended the previous results by adding two derived fields: *Cuml_Sales_Share* and *Cuml_PCT_Customers*. These fields represent the cumulative share of total revenue and the cumulative share of total customers, respectively.

This made it possible to identify what percentage of customers was required to reach a target revenue threshold. The completed view showed that both cumulative fields reached 100% (1.0) at the end of the ranked customer list, confirming that the calculations were complete.

Query results Chat Save results Open i

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	CustomerID	Cumlative_Custo...	Total_Customers	Cumlative_Reven...	Total_Revenue	Cuml_Sales_Share	Cuml_PCT_Custo...
253	1188	253	261	116254.95	116490.24	0.997980174133...	0.969348659003...
254	1016	254	261	116294.99	116490.24	0.998323893915...	0.973180076628...
255	1214	255	261	116333.27	116490.24	0.998652505136...	0.977011494252...
256	1172	256	261	116369.62	116490.24	0.998964548446...	0.980842911877...
257	1170	257	261	116404.92	116490.24	0.999267578124...	0.984674329501...
258	1131	258	261	116427.41	116490.24	0.999460641509...	0.988505747126...
259	1165	259	261	116448.95	116490.24	0.999645549704...	0.992337164750...
260	1075	260	261	116470.4	116490.24	0.999829685302...	0.996168582375...
261	1111	261	261	116490.24	116490.24	1.0	1.0

Figure 10: Cuml_Sales_Share and Cuml_PCT_Customers reached 100% (1.0).



Row	CustomerID	Cumlative_Custo...	Total_Customers	Cumlative_Reven...	Total_Revenue	Cuml_Sales_Share	Cuml_PCT_Custo...
146	1190	146	261	91650.23	116490.24	0.786763165738...	0.559386973180...
147	1173	147	261	92054.54	116490.24	0.790233928610...	0.563218390804...
148	1195	148	261	92458.81	116490.24	0.793704348106...	0.567049808429...
149	1028	149	261	92854.5	116490.24	0.797101113363...	0.570881226053...
150	1167	150	261	93249.3	116490.24	0.800490238495...	0.574712643678...

Figure 11: We can also see now that about 80% of sales come from roughly 57% of customers.

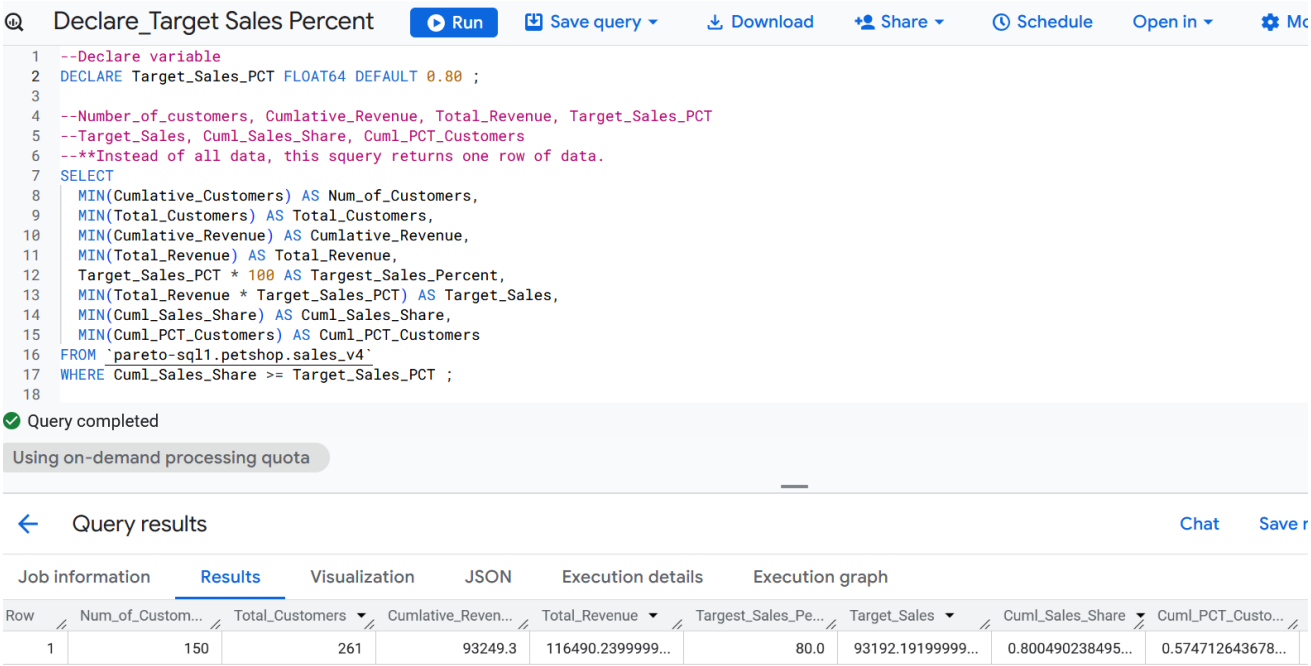
Target Threshold Analysis

A declared variable query was then used to make the threshold selection dynamic. The variable *Target_Sales_PCT* was set to a default value, such as 0.80 (80%), and the query returned the first row where cumulative sales share was greater than or equal to the selected target.

This approach allowed the same logic to be reused for different Pareto thresholds without rewriting the full query.

80% Revenue Threshold:

When *Target_Sales_PCT* was set to 0.80, the analysis showed that 150 out of 261 customers were required to reach approximately 80% of total revenue. This corresponds to roughly 57% of customers.



Row	Num_of_Custom...	Total_Customers	Cumlative_Reven...	Total_Revenue	Targest_Sales_Pe...	Target_Sales	Cuml_Sales_Share	Cuml_PCT_Custo...
1	150	261	93249.3	116490.2399999...	80.0	93192.19199999...	0.800490238495...	0.574712643678...

Figure 12: Query and results of cumulative percent of customers from 80% of the sales. 150/261 = 50%

This result indicates that revenue is concentrated, but not to the extent of a classic 80/20 distribution. A comparatively broad portion of the customer base is needed to generate most revenue, which suggests that revenue dependency is spread across more than just a narrow group of top customers.

50% Revenue Threshold:

When *Target_Sales_PCT* was set to 0.50(50%), the analysis showed that 79 out of 261 customers accounted for about 50% of total revenue. This corresponds with roughly 30% of customers.

This finding shows a moderate concentration pattern, where less than one-third of customers generate half of total sales. It supports the idea that higher-value customers are important, but not exclusively responsible for overall performance.

Declare_Target Sales Percent

Run | Save query | Download | Share | Schedule | Open in | M

```
1  --Declare variable
2  DECLARE Target_Sales_PCT FLOAT64 DEFAULT 0.50 ;
3
4  --Number_of_customers, Cumlative_Revenue, Total_Revenue, Target_Sales_PCT
5  --Target_Sales, Cuml_Sales_Share, Cuml_PCT_Customers
6  --**Instead of all data, this squery returns one row of data.
7  SELECT
8    MIN(Cumlative_Customers) AS Num_of_Customers,
9    MIN(Total_Customers) AS Total_Customers,
10   MIN(Cumlative_Revenue) AS Cumlative_Revenue,
11   MIN(Total_Revenue) AS Total_Revenue,
12   Target_Sales_PCT * 100 AS Targest_Sales_Percent,
13   MIN(Total_Revenue * Target_Sales_PCT) AS Target_Sales,
14   MIN(Cuml_Sales_Share) AS Cuml_Sales_Share,
15   MIN(Cuml_PCT_Customers) AS Cuml_PCT_Customers
16 FROM `pareto-sql1.petshop.sales_v4`
17 WHERE Cuml_Sales_Share >= Target_Sales_PCT ;
18
```

Query completed

Using on-demand processing quota

Query results | Chat | Save

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	Num_of_Custom...	Total_Customers	Cumlative_Reven...	Total_Revenue	Targest_Sales_Pe...	Target_Sales	Cuml_Sales_Share	Cuml_PCT_Custo...
1	79	261	58380.13	116490.2399999...	50.0	58245.11999999...	0.501158981215...	0.302681992337...

Figure 13: Query and results of cumulative percent of customers from 50% of the sales. 79/261 = 30%

20% Revenue Threshold:

When *Target_Sales_PCT* was set to 0.20(20%), the analysis showed that 28 out of 261 customers accounted for approximately 20% of total revenue. This corresponds with about 11% of customers.

This is the most concentrated portion of the revenue distribution and highlights a smaller group of customers with disproportionately high value. These customers may represent a useful segment for retention analysis, loyalty initiatives, or more detailed behavioral review.

Declare_Target Sales Percent

Run | Save query | Download | Share | Schedule | Open in

```sql
--Declare variable
DECLARE Target_Sales_PCT FLOAT64 DEFAULT 0.20 ;

--Number_of_customers, Cumlative_Revenue, Total_Revenue, Target_Sales_PCT
--Target_Sales, Cuml_Sales_Share, Cuml_PCT_Customers
--**Instead of all data, this squery returns one row of data.
SELECT
  MIN(Cumlative_Customers) AS Num_of_Customers,
  MIN(Total_Customers) AS Total_Customers,
  MIN(Cumlative_Revenue) AS Cumlative_Revenue,
  MIN(Total_Revenue) AS Total_Revenue,
  Target_Sales_PCT * 100 AS Targest_Sales_Percent,
  MIN(Total_Revenue * Target_Sales_PCT) AS Target_Sales,
  MIN(Cuml_Sales_Share) AS Cuml_Sales_Share,
  MIN(Cuml_PCT_Customers) AS Cuml_PCT_Customers
FROM `pareto-sql1.petshop.sales_v4`
WHERE Cuml_Sales_Share >= Target_Sales_PCT ;

--There are 150 customers, with a cumlative revenue of 93 249,30, who purchase 80% of the Total Revenue (116 490,24) of the Petshop.
--Cumlative_Revenue / Total_Revenue = Cuml_Sales_Share
```

This script will process 35.16 KB when run.

Query results | Chat | Save

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	Num_of_Custom...	Total_Customers	Cumlative_Reven...	Total_Revenue	Targest_Sales_Pe...	Target_Sales	Cuml_Sales_Share	Cuml_PCT_Custo...
1	28	261	23947.71	116490.2400000...	20.0	23298.04800000...	0.205576965074...	0.107279693486...

Figure 14: Figure 13: Query and results of cumulative percent of customers from 20% of the sales. 28/261 = 11%

Conclusion

This project used SQL in Google BigQuery to perform a Pareto analysis of customer revenue within a pet shop dataset. The analysis found that approximately 20% of sales came from 11% of customers, 50% of sales came from 30% of customers, and 80% of sales came from 57% of customers.

These results show that customer revenue is unevenly distributed, but the concentration is less extreme than a strict 80/20 pattern. In practical terms, this means that while high-spending customers contribute disproportionately to total revenue, a relatively broad customer segment is still required to generate most of the business's sales.

Overall, the analysis demonstrates how SQL window functions and cumulative share calculations can be used to quantify revenue concentration and support data-driven prioritization. The findings provide a useful basis for further work in customer segmentation, retention strategy, and revenue risk analysis.

Possible Next Steps

Several follow-up analyses would strengthen the business value of this project:

- Segment customers by country to determine whether revenue concentration differs across geographic markets.
- Analyze product or product-category revenue concentration to identify whether a small number of items drives a large share of total sales.
- Compare Pareto patterns across time periods to assess whether customer concentration is stable or changing over time.
- Investigate top-spending customers in more detail to identify repeat purchase behavior, retention opportunities, or patterns associated with high-value accounts.

Query Data Dictionary:

Field name	Type	Query	Description
Num_of_Customers	Integer	MIN(Cumulative_Customers)	Total number of customers
Total_Customers	Integer	COUNT(*) OVER()	Counts how many customer rows are in the result set and repeats that total on every row.
Cumulative_Revenue	Integer	SUM(CustomerRevenue) OVER(ORDER BY CustomerRevenue DESC ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)	Computes a running total of CustomerRevenue from the highest spender to the lowest spender.
Total_Revenue	Integer	SUM(CustomerRevenue) OVER()	Computes the total revenue across all customers and repeats that same value on every row.
CustomerRevenue	Integer	SUM(Sales)	The total sales value per customer.
Sales	Integer	(Quantity * UnitPrice)	From the original dataset, the quantity of the stock item multiplied by its unit price.

Cumulative_Customers	Integer	ROW_NUMBER() OVER(ORDER BY CustomerRevenue DESC)	Window function to retain row level integrity. Assigns a unique sequential number to each row, starting at 1 for the highest CustomerRevenue, 2 for the next, etc. This is effectively the rank by revenue with no ties allowed (even equal revenue values get different row numbers)
Cuml_Sales_Share	Integer	Cumulative_Revenue / Total_Revenue	Percentage of sales threshold.
Cuml_PCT_Customers	Integer	Cumulative_Customers / Total_Customers	Cumulative percentage of customers.
Target_Sales_PCT	Integer	DECLARE Target_Sales_PCT FLOAT64 DEFAULT 0.80 ;	Declares the target cumulative sales threshold used in the query.
Targest_Sales_Percent	Integer	Target_Sales_PCT * 100	Converts the target sales threshold to percentage format.
Target_Sales	Integer	(Total_Revenue * Target_Sales_PCT)	Calculates the revenue value corresponding to the selected sales threshold.